

קורס NodeJS תשפו

מבוא

NodeJS היא סביבת ריצה לקוד javascript, המיועדת להרצת קוד בserver. NodeJS רצה על מנוע V8 של google chrome מחוץ לדפדפן. הוא open source וcross platform ולכן פופולרי מאד בשנים האחרונות בפיתוח מערכות backend מורכבות.

אפליקציה של node רצה על single process, בלי ליצור thread חדש לכל בקשה. כדי למנוע מהthread להיתקע על ביצוע פעולות כמו I/O, פעולות אלו מבוצעות בצורה אסינכרונית – הפעולה נשלחת לביצוע ע"י הnetwork, מערכת הקבצים, DB או כל ספק חיצוני אחר. כשביצוע הפעולה מסתיים, הוא חוזר לprocess להמשך הרצה של הקוד.

בעזרת המנגנון האסינכרוני הזה, אפליקציה אחת של nodejs שרצה על thread אחד, יכולה לנהל אלפי בקשות במקביל – בלי הנטל של ניהול מקביליות threads, מה שבטכנולוגיות אחרות מהווה לעיתים קרובות מקור לעומסים ולבאגים.

יתרונות

- **ביצועים גבוהים** - שימוש במנוע JavaScript V8 של גוגל, המהיר במיוחד בקימפול והרצת קוד. כמו כן, המנגנון האסינכרוני של nodejs המבוסס על event loop, מאפשר ריבוי משימות יעיל וביצועים גבוהים במשאבים נמוכים.
- כמו כן, הודות לריצה על process אחד, nodejs יעיל במיוחד בעבודה בעזרת microservices.
- **כתיבה בjavascript** – מפתחי frontend שמכירים את השפה יכולים לכתוב גם את צד השרת בקלות מבלי ללמוד שפה חדשה.
- **ספרית הרחבות גדולה** – ספריית ההרחבות של nodejs (NPM) מציעה מעל למיליון חבילות open source שניתן להשתמש בהן לפיתוח מהיר ויעיל.

התקנה

1. גשו לאתר הרשמי של Node.js: <https://nodejs.org>
2. הורידו את הגרסה האחרונה המתאימה למערכת ההפעלה.
3. הריצו את קובץ ההתקנה.
4. בדקו שההתקנה הצליחה על ידי הרצת הפקודה הבאה בcommand line:

```
node -v
```

פקודה זו אמורה להחזיר את גרסת Node.js שהותקנה.

דוגמת קוד בסיסית

הדוגמה הבאה מכילה קטע קוד שקורא input מתוך קובץ, וכותב הודעת output לקובץ אחר:

```
const { readFileSync, writeFileSync } = require("fs");

function welcomeName(inputFile, outputFile) {
  const name = readFileSync(inputFile, "utf-8");
  const message = `Hello, ${name}! Welcome to Node.js.`;

  writeFileSync(outputFile, message);

  console.log("Done!");
}
welcomeName("name.txt", "output.txt");
```

הסבר הקוד:

```
const { readFileSync, writeFileSync } = require("fs");
```

טוען את הפונקציות readFileSync, writeFileSync מתוך הספרייה fs (ספרייה שכלולה בקוד הבסיסי של node).

```
function welcomeName(inputFile, outputFile) {
```

מצהיר על הפונקציה welcomeName שמקבלת שני פרמטרים.

```
const name = readFileSync(inputFile, "utf-8");
```

קורא את תוכן הקובץ ששמו הועבר כפרמטר ראשון לפונקציה - inputFile, ומכניס אותו למשתנה name.

```
const message = `Hello, ${name}! Welcome to Node.js.`;
writeFileSync(outputFile, message);
```

כותב הודעת welcome הכוללת את השם שנקרא מקובץ inputFile לקובץ ששמו הועבר כפרמטר שני לפונקציה - outputFile.

```
console.log("Done!");
```

מדפיס למסך "Done!".

```
welcomeName("name.txt", "output.txt");
```

מזמן את הפונקציה welcomeName עם שני פרמטרים.

הרצה

כדי את הקוד יש לשמור את הקובץ בסיומת js ולהריץ בcommand line:

```
node <filename>
```

לדוגמא:

```
node app.js
```

כדי להריץ את דוגמת הקוד מלמעלה, נשמור את הקובץ בשם basic-node.js. ניצור קובץ חדש בשם name.txt ונכתוב בו שם כלשהו. נריץ את הסקריפט:

```
node basic-node.js
```

הoutput שיודפס לconsole:

```
Done!
```

נראה שנוצר קובץ חדש בשם output.txt שהתוכן שלו הוא:

```
Hello, <name>! Welcome to Node.js.
```

TypeScript

TypeScript היא שפה הבנויה על גבי javascript ומוסיפה לה types.

ניתן לקמפל בקלות את הקוד שנכתב בtypescript ולהמיר אותו לscript של javascript. בנוסף, ישנן ספריות להרצת typescript ישירות מבלי לקמפל קודם לקובץ javascript.

יתרונות:

- פיתוח נח ויעיל יותר.
- זיהוי באגים בזמן קומפילציה במקום בזמן ריצה.
- קוד קריא ומסודר.

במהלך הקורס כל הקוד שנכתוב יהיה בtypescript.

דוגמה לקוד TypeScript

```
function greet(name: string): string {  
    return `Hello, ${name}!`;  
}  
  
console.log(greet('Node.js'));
```

הסבר הקוד:

```
function greet(name: string): string
```

פונקציה שמקבלת מחרוזת (name) ומחזירה מחרוזת.

```
return `Hello, ${name}!`
```

החזרת string שכולל בתוכו את השם שהתקבל בargument של הפונקציה.

```
console.log(greet('Node.js'))
```

הדפסת הערך המוחזר מהפונקציה לconsole.

הרצת קוד typescript

כדי להריץ typescript נשתמש בספריה tsx המאפשרת הרצת קובץ typescript ישירות מה command line – מבלי צורך לקמפל.

להתקנת החבילה יש להריץ את הפקודה:

```
npm i -D tsx
```

להרצת הקובץ יש לשמור אותו בסיומת ts, ולהריץ:

```
npx tsx <filename>
```

לדוגמא:

```
npx tsx app.ts
```

דיבוג קוד typescript

כדי לדבג קוד typescript, נשתמש בקונפיגורציה של vscode שתקמפל תחילה את הקובץ ותמיר אותו לjs ואז תריץ את הjs שנוצר, תוך מיפוי breakpoints מקובץ הts לקובץ הjs.

יש ליצור קודם כל קובץ tsconfig.json שמכיל את הגדרות הקומפילציה:

```
{
  "compilerOptions": {
    "target": "ES5",
    "module": "CommonJS",
    "outDir": "out",
    "sourceMap": true
  }
}
```

כעת יש ליצור תיקיה בשם `.vscode`. ותחתיה קובץ בשם `launch.json` שמכיל את הגדרות ההרצה:

```
{
  "version": "1.0.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Launch Program",
      "program": "${workspaceFolder}/app.ts",
      "preLaunchTask": "tsc: build - tsconfig.json",
      "outFiles": ["${workspaceFolder}/out/**/*.js"]
    }
  ]
}
```

ניתן לשנות את `app.ts` לכל שם קובץ אחר כדי להריץ אותו.

לאחר יצירת הקבצים, אפשר להוסיף breakpoints בקובץ ה-`ts`, ולהריץ את הקוד דרך `Run and Debug`.
את ההדפסות ל-`console` ניתן לראות בחלונית `Debug Console`, דרכה גם אפשר להדפיס ערכים של משתנים בעת העצירה ב-`breakpoint`.

Types

```
let age: number = 25;
let name: string = "John";
let isStudent: boolean = true;
```

Functions with types

```
function add(a: number, b: number): number {
    return a + b;
}
```

Interfaces

```
interface User {
    id: number;
    name: string;
}

const user: User = { id: 1, name: "Alice" };
```

Classes

```
class Person {
    constructor(public name: string, public age: number) {}
    greet() {
        return `Hello, my name is ${this.name}`;
    }
}

const person = new Person("David", 30);
console.log(person.greet());
```

Generics

```
function identity<T>(arg: T): T {
    return arg;
}

console.log(identity<number>(5));
console.log(identity<string>("Hello"));
```